

Tugas Lab. Algo Pertemuan 13

Nama : Fathurrohman

NPM : 252310040

Kelas : TI-25-PA2

Github: <https://github.com/Fathur123-TI/LabAlgoritmaSemester2.git>

Pengertian Hashing

Hashing adalah salah satu metode dalam struktur data yang digunakan untuk menyimpan, mencari, dan mengelola data dengan lebih cepat. Cara kerja hashing yaitu dengan mengubah sebuah nilai kunci (key) menjadi sebuah alamat penyimpanan (index) menggunakan fungsi hash (hash function). Hasil dari fungsi hash tersebut digunakan untuk menentukan lokasi data di dalam Hash Table.

Tujuan hashing adalah untuk membuat proses penyimpanan dan pencarian data menjadi lebih cepat dan mudah dengan menggunakan key untuk menentukan lokasi penyimpanan data pada Hash Table.

Hashing dibagi menjadi 2 jenis utama, yaitu:

1. Open Hashing (Separate Chaining)

Open Hashing adalah metode hashing yang menangani collision dengan cara menyimpan beberapa data pada index yang sama. Jika terdapat data yang memiliki hasil hash yang sama, maka data tersebut akan disimpan menggunakan struktur tambahan seperti linked list pada index tersebut.

2. Closed Hashing (Open Addressing)

Closed Hashing adalah metode hashing yang menangani collision dengan cara mencari tempat kosong lain di dalam Hash Table. Jika index hasil hash sudah terisi, maka program akan mencari lokasi berikutnya sampai menemukan slot kosong. Metode ini menggunakan teknik seperti **Linear Probing, Quadratic Probing, dan Double Hashing**

```
#include <iostream>
// Library iostream digunakan untuk proses input dan output.
// Contohnya cin untuk menerima input dari pengguna dan cout untuk menampilkan output.
```

```
#include <string>
// Library string digunakan agar program dapat menyimpan data berupa teks,
// seperti nama barang pada inventory.
```

```
using namespace std;
// Agar kita dapat menggunakan cin, cout, string tanpa harus menulis std:: di depannya.
```

```
// Membuat tipe data enum bernama Status.
// Enum digunakan untuk memberikan tanda kondisi pada setiap slot Hash Table.
// Setiap tempat penyimpanan dapat memiliki tiga kondisi:
// EMPTY = slot masih kosong
// FILLED = slot sudah berisi data
// DELETED = data pernah dihapus
enum Status {
    EMPTY,
    FILLED,
    DELETED
};
```

```
// Membuat struktur data Item.
```

```

// Struktur ini digunakan untuk menyimpan informasi barang yang dimasukkan
// ke dalam Hash Table.
struct Item {

    string nama;
    // Variabel nama digunakan untuk menyimpan nama barang.

    int key;
    // Variabel key digunakan sebagai nilai kunci yang menentukan
    // posisi penyimpanan data pada Hash Table.

    Status status;
    // Variabel status digunakan untuk mengetahui kondisi slot
    // apakah kosong, berisi, atau sudah dihapus.

    Item() {
        // Constructor digunakan untuk memberikan nilai awal
        // ketika objek Item dibuat.

        nama = "";
        // Nama barang awal dibuat kosong.
        key = 0;
        // Nilai key awal diberikan 0.

        status = EMPTY;
        // Semua slot Hash Table pertama kali dianggap kosong.
    }
};

// Membuat pointer hashTable untuk menunjuk ke lokasi memori
// yang digunakan sebagai tempat penyimpanan tabel Hash.
Item *hashTable;

// Variabel SIZE digunakan untuk menyimpan jumlah kapasitas
// atau ukuran dari Hash Table.
int SIZE;

// Fungsi untuk mengecek apakah sebuah angka merupakan bilangan prima.
// Ukuran Hash Table biasanya menggunakan bilangan prima agar
// penyebaran data menjadi lebih baik dan mengurangi kemungkinan benturan data.
bool isPrima(int n) {

    if(n <= 1)
        // Angka 1 dan angka negatif bukan termasuk bilangan prima.
        return false;

    for(int i = 2; i * i <= n; i++) {
        // Melakukan pengecekan pembagi mulai dari angka 2.
        // Jika ditemukan pembagi yang tepat, maka angka tersebut bukan prima.

        if(n % i == 0)

```

```

        return false;
    }

    return true;
    // Jika tidak ditemukan pembagi, berarti angka tersebut prima.
}

// Fungsi untuk mencari bilangan prima berikutnya.
// Fungsi ini digunakan jika ukuran tabel yang dimasukkan pengguna
// bukan bilangan prima.
int nextPrima(int n) {

    while(!isPrima(n)) {
        // Selama angka belum menjadi prima,
        // nilai angka akan terus ditambah.

        n++;
    }

    return n;
    // Mengembalikan nilai bilangan prima yang ditemukan.
}

// Fungsi Hash.
// Fungsi ini digunakan untuk menentukan index awal penyimpanan data.
// Rumus yang digunakan adalah key modulo ukuran tabel.
int hashFunction(int key) {

    return key % SIZE;
}

// Fungsi untuk memasukkan data barang ke dalam Hash Table.
void insertItem() {

    string nama;
    // Variabel untuk menyimpan nama barang.

    int key;
    // Variabel untuk menyimpan kode/key barang.

    cout << "\nMasukkan nama item : ";
    cin >> nama;

    cout << "Masukkan key item : ";
    cin >> key;

    int index = hashFunction(key);
    // Menentukan lokasi awal penyimpanan berdasarkan fungsi hash.

    int awal = index;
    // Menyimpan posisi awal agar dapat mengetahui

```

```

// apakah pencarian sudah kembali ke titik awal.

while(hashTable[index].status == FILLED) {
    // Jika posisi tersebut sudah berisi data,
    // maka terjadi collision (tabrakan data).

    // Penyelesaian collision menggunakan Linear Probing,
    // yaitu mencari slot kosong berikutnya.

    index = (index + 1) % SIZE;

    if(index == awal) {
        // Jika kembali ke posisi awal,
        // berarti seluruh tabel sudah penuh.

        cout << "Hash Table penuh!\n";
        return;
    }
}

hashCode[index].nama = nama;
// Menyimpan nama barang ke dalam slot.

hashCode[index].key = key;
// Menyimpan key barang.

hashCode[index].status = FILLED;
// Mengubah status slot menjadi berisi data.

cout << "Item berhasil disimpan pada index "
    << index << endl;
}

// Fungsi untuk menghapus data berdasarkan key.
void hapusItem() {

    int key;

    cout << "\nMasukkan key yang ingin dihapus : ";
    cin >> key;

    int index = hashCode(key);
    // Menentukan posisi awal pencarian.

    int awal = index;

    while(hashTable[index].status != EMPTY) {
        // Pencarian dilakukan selama slot tidak kosong.

        if(hashTable[index].status == FILLED &&

```

```

    hashTable[index].key == key) {

        // Jika key ditemukan,
        // status data diubah menjadi DELETED.

        hashTable[index].status = DELETED;

        cout << "Item berhasil dihapus.\n";
        return;
    }

    index = (index + 1) % SIZE;

    if(index == awal)
        break;
}

cout << "Item tidak ditemukan.\n";
}

// Fungsi untuk mencari data berdasarkan key.
void lihatItem() {

    int key;

    cout << "\nMasukkan key yang dicari : ";
    cin >> key;

    int index = hashFunction(key);
    int awal = index;

    while(hashTable[index].status != EMPTY) {

        if(hashTable[index].status == FILLED &&
            hashTable[index].key == key) {

            cout << "\nData ditemukan\n";
            cout << "Index : " << index << endl;
            cout << "Nama : " << hashTable[index].nama << endl;
            cout << "Key : " << hashTable[index].key << endl;

            return;
        }

        index = (index + 1) % SIZE;

        if(index == awal)
            break;
    }
}

```

```

    cout << "Data tidak ditemukan.\n";
}

// Fungsi untuk menampilkan seluruh data yang tersimpan.
void tampilkanSemua() {

    cout << "\nSemua Data Inventory\n";

    bool ada = false;
    // Variabel penanda untuk mengecek apakah tabel memiliki data.

    for(int i = 0; i < SIZE; i++) {

        // Melakukan pengecekan seluruh index Hash Table.

        if(hashTable[i].status == FILLED) {

            cout << "\nIndex : " << i << endl;
            cout << "Nama : " << hashTable[i].nama << endl;
            cout << "Key : " << hashTable[i].key << endl;
            cout << "-----\n";
            ada = true;
        }
    }

    if(!ada)
        cout << "Data masih kosong.\n";
}

// Fungsi utama program.
int main() {

    int jumlahData;

    cout << "Masukkan jumlah data : ";
    cin >> jumlahData;

    if(!isPrima(jumlahData)) {

        // Jika ukuran tabel bukan bilangan prima,
        // program akan mencari bilangan prima berikutnya.

        SIZE = nextPrima(jumlahData);

        cout << "Ukuran Hash Table diubah menjadi "
            << SIZE << endl;

    }
    else {

        SIZE = jumlahData;
    }
}

```

```

    // Jika sudah prima, ukuran langsung digunakan.
}

hashTable = new Item[SIZE];
// Membuat Hash Table secara dinamis sesuai ukuran.

int pilihan;

do {

    cout << "\nInventory Manajemen Sederhana\n";
    cout << "1. Insert Item\n";
    cout << "2. Hapus Item\n";
    cout << "3. Cari/Lihat Item\n";
    cout << "4. Tampilkan Semua Item\n";
    cout << "0. Keluar\n";

    cout << "Pilih menu : ";
    cin >> pilihan;

    switch(pilihan) {

        case 1:
            insertItem();
            break;

        case 2:
            hapusItem();
            break;

        case 3:
            lihatItem();
            break;

        case 4:
            tampilkanSemua();
            break;

        case 0:
            cout << "Program selesai.\n";
            break;

        default:
            cout << "Pilihan tidak tersedia.\n";
    }

} while(pilihan != 0);

delete[] hashTable;
// Menghapus memori yang digunakan oleh Hash Table.

```

```
    return 0;
    // Program selesai.
}jarak spasinya bisa lebih dipadetin gk
```

Output:

```
Masukkan jumlah data : 100
Ukuran Hash Table diubah menjadi 101

Inventory Manajemen Sederhana
1. Insert Item
2. Hapus Item
3. Cari/Lihat Item
4. Tampilkan Semua Item
0. Keluar
Pilih menu : 1
Masukkan nama item : Komputer
Masukkan key item : 203
Item berhasil disimpan pada index 1

Inventory Manajemen Sederhana
1. Insert Item
2. Hapus Item
3. Cari/Lihat Item
4. Tampilkan Semua Item
0. Keluar
Pilih menu : 0
Program selesai.

-----
Process exited after 34.29 seconds with return value 0
Press any key to continue . . . |
```